

Hashing

```

const int M = 2 , N = 1e6 + 5;
array<int,2> B , MODS , pw [N];
void build() {
    if (B[0]) return;

    random_device rd;
    mt19937 mt(rd());

    auto rnd = [&](int l, int r) {
        return uniform_int_distribution<int>(l, r)(mt);
    };

    auto is_prime = [](int x) {
        if (x < 2) return false;
        for (int i = 2; i * i <= x; ++i)
            if (x % i == 0) return false;
        return true;
    };

    B[0] = rnd(100, 500);
    B[1] = rnd(100, 500);

    MODS[0] = rnd(2e8, 2e9);
    MODS[1] = rnd(2e8, 2e9);

    while (!is_prime(MODS[0])) MODS[0]--;
    while (!is_prime(MODS[1]) || MODS[0] == MODS[1]) MODS[1]--;

    for (int j = 0; j < M; ++j) {
        pw[0][j] = 1;
        for (int i = 1; i < N; ++i) {
            pw[i][j] = (1LL * pw[i-1][j] * B[j]) % MODS[j];
        }
    }
}

struct Hash
{
    vector<array<int,M>>pref , rev ;

    void initPref (string &s){
        int n = s.size() ;
        pref.assign(n,{}) ;
        array<int,2> h {} ;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < M; j++)
            {

```

```

        h[j] = (h[j] * B[j]) % MODS[j];
        h[j] = (h[j] + s[i]) % MODS[j];
    }
    pref[i] = h ;
}
}
void initSuff (string &s){
    int n = s.size() ;
    rev.assign(n,{'}) ;
    array<int,2> h {} ;
    for (int i = n-1; i >= 0; i++)
    {
        for (int j = 0; j < M; j++)
        {
            h[j] = (h[j] * B[j]) % MODS[j];
            h[j] = (h[j] + s[i]) % MODS[j];
        }
        rev[i] = h ;
    }
}
array<int , M> substr (int l , int r){
    if(!l) return pref[r] ;
    auto res = pref[r] ;
    for (int j = 0; j < M; j++)
    {
        res[j] -= (1LL * pw[r-l+1][j] * pref[l-1][j]) % MODS[j] ;
        if(res[j] < 0) res[j]+= MODS[j] ;
    }
    return res ;
}
array<int , M> substrRev (int l , int r){
    if(r == rev.size()-1) return rev[l] ;
    auto res = rev[l] ;
    for (int j = 0; j < M; j++)
    {
        res[j] -= (1LL * pw[r-l+1][j] * rev[l-1][j]) % MODS[j] ;
        if(res[j] < 0) res[j]+= MODS[j] ;
    }
    return res ;
}
bool isPal (int l , int r){
    return substr(l , r) == substrRev(l , r) ;
}
};

```

Hashing Template Better :

```
const int N = 1e6 + 5;
```

```

const int MODS[] = {1000000000 + 7, 1000000000 + 9, 1000000000 + 21, 1000000000 +
33, 1000000000 + 87};
mt19937_64 rng(chrono::system_clock::now().time_since_epoch().count());
int MOD = MODS[uniform_int_distribution<int>(0, 4)(rng)];
int BASE = uniform_int_distribution<int>(256, MOD - 2)(rng);
int pw[N];
void build() {
    pw[0] = 1;
    for (int i = 1; i < N; i++)
        pw[i] = 1LL * pw[i - 1] * BASE % MOD;
}
struct PrefixHash {
    vector<int> pref;

    void init(const string &s) {
        int n = s.size();
        pref.assign(n + 1, 0);
        for (int i = 0; i < n; i++)
            pref[i + 1] = (1LL * pref[i] * BASE + s[i]) % MOD;
    }

    int get(int l, int r) {
        int res = pref[r + 1] - 1LL * pref[l] * pw[r - l + 1] % MOD;
        if (res < 0) res += MOD;
        return res;
    }
};

struct SuffixHash {
    vector<int> suff;

    void init(const string &s) {
        int n = s.size();
        suff.assign(n + 1, 0);
        for (int i = n - 1; i >= 0; i--)
            suff[i] = (1LL * suff[i + 1] * BASE + s[i]) % MOD;
    }

    int get(int l, int r) {
        int res = suff[l] - 1LL * suff[r + 1] * pw[r - l + 1] % MOD;
        if (res < 0) res += MOD;
        return res;
    }
};

```

XOR Hashing :

```

mt19937_64 rng(chrono::system_clock::now().time_since_epoch().count());

```

```
int rnd(int a, int b) {  
    if (a > b) return 0;  
    return a + rng() % (b - a + 1);  
}
```

manacher Longest plaindromic substring in O(n) time complexity

```
// Longest plaindromic substring in O(n) time complexity  
string manacher(string s) {  
    string arr;  
    for (char c : s) {  
        arr.push_back('#');  
        arr.push_back(c);  
    }  
    arr.push_back('#');  
  
    int n = arr.size();  
    vector<int> dp(n, 0);  
    int center = 0, right = 0;  
    int maxLen = 0, maxCenter = 0;  
  
    for (int i = 0; i < n; ++i) {  
        int mirror = 2 * center - i;  
        if (i < right)  
            dp[i] = min(right - i, dp[mirror]);  
  
        int a = i + (dp[i] + 1);  
        int b = i - (dp[i] + 1);  
  
        while (a < n && b >= 0 && arr[a] == arr[b]) {  
            dp[i]++;  
            a++;  
            b--;  
        }  
  
        if (i + dp[i] > right) {  
            center = i;  
            right = i + dp[i];  
        }  
  
        if (dp[i] > maxLen) {  
            maxLen = dp[i];  
            maxCenter = i;  
        }  
    }  
  
    string res = "";  
    for (int i = maxCenter - maxLen; i <= maxCenter + maxLen; ++i) {
```

```
        if (arr[i] != '#')
            res += arr[i];
    }
    return res;
}
```

Z algorithm

```
vector<int> computeZ(string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; ++i) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            ++z[i];
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

KMP ALL

```
string s, p;
cin >> s >> p;

int n = s.length();
int m = p.length();

// Build prefix function
vector<int> pi(m);
for (int i = 1; i < m; i++) {
    int j = pi[i - 1];
    while (j > 0 && p[i] != p[j])
        j = pi[j - 1];
    if (p[i] == p[j])
        j++;
}
```

```

    pi[i] = j;
}

// Build nxt array: prefix automaton transitions
vector<vector<int>> nxt(m + 1, vector<int>(26));
for (int i = 0; i <= m; i++) {
    for (int c = 0; c < 26; c++) {
        if (i < m && p[i] == char('a' + c))
            nxt[i][c] = i + 1;
        else if (i > 0)
            nxt[i][c] = nxt[pi[i - 1]][c];
        else
            nxt[i][c] = 0;
    }
}

// KMP search to find all matches of p in s
vll ans;
for (int i = 0, k = 0; i < n; i++) {
    while (k > 0 && p[k] != s[i])
        k = pi[k - 1];
    if (p[k] == s[i])
        k++;
    if (k == m) {
        ans.push_back(i - m + 2); // 1-based position
        k = pi[k - 1];
    }
}

int count_distinct_substrings(string s) {
    int n = s.length();
    int total_distinct = 0;
    string current_str = "";

    for (int i = 0; i < n; i++) {
        current_str += s[i];

        string reversed_str = current_str;
        reverse(reversed_str.begin(), reversed_str.end());

        vector<int> pi = compute_prefix_function(reversed_str);

        int pi_max = 0;
        for (int val : pi) {
            pi_max = max(pi_max, val);
        }

        int new_substrings = current_str.length() - pi_max;
        total_distinct += new_substrings;
    }

    return total_distinct;
}

```

Aho Corasick

```
struct Node {
    unordered_map<char, int> ch;
    int fail{}, nxt{}, id = -1;
};

struct Aho {

    static const int ALPHA = 128;
    vector<Node> Trie;

    int addNode() {
        Trie.emplace_back();
        return Trie.size() - 1;
    }

    void init() {
        Trie.clear();
        addNode();
    }

    Aho() { init(); }

    /// If the same string appears before, return its index
    int insert(const string &pat, int ID) {
        int u = 0;
        for (char c: pat) {
            int &nd = Trie[u].ch[c];
            if (!nd) nd = addNode();
            u = nd;
        }
        int &id = Trie[u].id;
        return ~id ? id : (id = ID);
    }

    int nxtF(int u, char c) {
        while (!Trie[u].ch.count(c))
            u = Trie[u].fail;
        u = Trie[u].ch[c];
        return u;
    }

    int Nxt(int u) {
        if (!u) return u;
        int &v = Trie[u].nxt;
        return ~Trie[v].id ? v : v = Nxt(v);
    }
}
```

```

void computeFail() {
    queue<int> q;
    for (int i = 0; i < ALPHA; i++) {
        int &nd = Trie[0].ch[i];
        if (nd)
            q.push(nd);
    }
    while (q.size()) {
        int u = q.front();
        q.pop();
        int f = Trie[u].fail;
        for (auto [ch, v]: Trie[u].ch) {
            Trie[v].fail = Trie[v].nxt = nxtF(f, ch);
            q.push(v);
        }
    }
}

vector<vector<int>> match(string &s, int numPat) {
    vector<vector<int>> ret(numPat);
    int cur = 0, i = 0;
    for (char c: s) {
        cur = nxtF(cur, c);
        for (int p = cur; p; p = Nxt(p)) {
            int &x = Trie[p].id;
            if (~x) ret[x].push_back(i);
        }
        ++i;
    }
    return ret;
}

};

```

Suffix Array

```

struct SuffixArray {
#define vi vector<int>
#define vvi vector<vi>
#define pii pair<int,int>
    const int lim = 128;
    int n;
    vi sa, lcp;
    vvi c, lcpQ;

    SuffixArray(string s) {
        constructSA(s);
        constructLCP(s);
    }
}

```

```

void constructSA(string s) {
    // a sorted permutation suffixes
    // the equivalence classes of each suffix
    // cnt of each equivalence class
    // the permutation and equivalence classes of new iteration
    vi p, cnt, pn;
    s += '$';
    n = (int) s.size();
    int LG = __lg(n) + 2;
    c = vvi(LG, vi(n));
    p.resize(n);
    pn.resize(n);
    cnt.resize(max(lim, n));

    //perform counting sort on suffixes based on one character
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < lim; i++)
        cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--)
        p[--cnt[s[i]]] = i;

    //calculate equivalence classes
    int classes = 1;
    c[0][p[0]] = classes - 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]])
            ++classes;
        c[0][p[i]] = classes - 1;
    }

    //repeat above procedure but for substrings of length (1 << (h+1))
    for (int h = 0; (1 << h) < n; ++h) {
        //sort based on the second part
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }

        fill(cnt.begin(), cnt.begin() + classes, 0);
        //sort based on the first part
        for (int i = 0; i < n; i++)
            cnt[c[h][pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--)
            p[--cnt[c[h][pn[i]]]] = pn[i];

        classes = 1;
        c[h + 1][p[0]] = classes - 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[h][p[i]], c[h][(p[i] + (1 << h)) % n]};

```

```

        pair<int, int> prev = {c[h][p[i - 1]], c[h][(p[i - 1] + (1 << h)) % n]};

        if (cur != prev)
            ++classes;
        c[h + 1][p[i]] = classes - 1;
    }
}
sa = p;
}

void constructLCP(string s) {
    //lcp between two adjacent suffixes in the suffix array
    lcp.resize(n, 0);
    // rank of the i-th suffix (by length)
    vi rnk(n);
    for (int i = 0; i < n; i++)
        rnk[sa[i]] = i;

    int k = 0;
    //iterating over suffixes by length
    for (int i = 0; i < n; i++) {
        if (rnk[i] == n - 1) {
            k = 0;
            continue;
        }

        int j = sa[rnk[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k])
            ++k;
        lcp[rnk[i]] = k;
        if (k)
            --k;
    }

    int K = __lg(n) + 1;
    lcpQ.resize(K, vi(n));
    for (int l = 0; l < K; l++) {
        for (int i = 0; i + (1 << l) - 1 < n; i++)
            if (l == 0) lcpQ[l][i] = lcp[i];
            else lcpQ[l][i] = min(lcpQ[l - 1][i], lcpQ[l - 1][i + (1 << (l - 1))]);
    }
}

int queryLCP(int l, int r) {
    if (l == r) return n - sa[l] - 1;
    --r;
    int len = r - l + 1;
    int k = __lg(len);
    return min(lcpQ[k][l], lcpQ[k][r - (1 << k) + 1]);
}

int compare(int i, int j, int len) {
    int lg = __lg(len);
    pii a = {c[lg][i], c[lg][(i + len - (1 << lg)) % n]};

```

```

pii b = {c[lg][j], c[lg][(j + len - (1 << lg)) % n]};

if (a < b)
    return -1;
else if (a > b)
    return 1;
else
    return 0;
}

int getLCP(int i, int j) {
    int ans = 0;
    for (int k = __lg(n); k >= 0; k--) {
        if (c[k][i] == c[k][j]) {
            ans += (1 << k);
            i = (i + (1 << k)) % n;
            j = (j + (1 << k)) % n;
        }
    }
    return ans;
}

// optimize later
bool subStringSearch(string &s, string &pat) {
    const int m = pat.size();
    int l = 0, r = sa.size() - 1;
    while (l <= r) {
        int md = (l + r) / 2;
        string t = s.substr(sa[md], min(n - sa[md], m));
        if (t == pat) return true;
        else if (t < pat) l = md + 1;
        else r = md - 1;
    }
    return false;
}

ll lowerBound(string &s, string &pat) {
    const int m = pat.size();
    int l = 0, r = sa.size() - 1;
    while (l <= r) {
        int md = (l + r) / 2;
        string t = s.substr(sa[md], min(n - sa[md], m));
        if (t < pat) l = md + 1;
        else r = md - 1;
    }
    return r+1;
}

ll upperBound(string &s, string &pat) {
    const int m = pat.size();
    int l = 0, r = sa.size() - 1;
    while (l <= r) {
        int md = (l + r) / 2;
        string t = s.substr(sa[md], min(n - sa[md], m));

```

```

        if (t <= pat) l = md + 1;
        else r = md - 1;
    }
    return l-1;
}

```

```

#undef pii
#undef vvi
#undef vi
};

```

Suffix Automaton

```

struct SuffixAutomaton {
#define ll long long
#define ii pair<int,int>
#define S second
#define F first
    static const int N = 1e5 + 5;
    int last = 0, cntState = 1;
    struct state {
        int len = 0, link = -1, cnt = 0;
        map<char, int> next;
    };

    state st[N * 2];
    ll dp[N * 2];

    void addChar(char c) {
        int p = last;
        int cur = cntState++;
        st[cur].len = st[p].len + 1;
        st[cur].cnt = 1;
        while (p != -1 && st[p].next.count(c) == 0) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[q].len == st[p].len + 1) {
                st[cur].link = q;
            } else {
                int clone = cntState++;
                st[clone].link = st[q].link;
                st[clone].len = st[p].len + 1;
            }
        }
    }
}

```

```

        st[q].link = st[cur].link = clone;
        st[clone].next = st[q].next;
        st[clone].cnt = 0;
        while (p != -1 && st[p].next[c] == q) {
            st[p].next[c] = clone;
            p = st[p].link;
        }
    }
}
last = cur;
}

SuffixAutomaton() {}

SuffixAutomaton(const string &s) { for (char c: s) addChar(c); }

bool checkForOccurrence(const string &str) {
    int cur = 0;
    for (char i: str) {
        if (st[cur].next.count(i)) {
            cur = st[cur].next[i];
        } else {
            return false;
        }
    }
    return true;
}

ll numberOfSubstrings() {
    ll ans = 0;
    for (int i = 1; i < cntState; i++) {
        ans += st[i].len - st[st[i].link].len;
    }
    return ans;
}

ll lengthOfSubstrings() {
    auto getSum = [&](ll x) -> ll {
        return x * (x + 1) / 2;
    };
    ll ans = 0;
    for (int i = 1; i < cntState; i++) {
        ans += getSum(st[i].len) - getSum(st[st[i].link].len);
    }
    return ans;
}

void numberOfOccPreProcess() {
    static bool visisted = false;
    if (visisted) return;
    visisted = true;
    vector<ii> v;
    for (int i = 0; i < cntState; i++)
        v.emplace_back(st[i].len, i);
}

```

```

    sort(v.rbegin(), v.rend());
    for (int i = 0; i < cntState; i++) {
        if (st[v[i].S].link > 0)
            st[st[v[i].S].link].cnt += st[v[i].S].cnt;
        dp[v[i].S] = st[v[i].S].cnt;
        for (auto x: st[v[i].S].next) {
            dp[v[i].S] += dp[x.S];
        }
    }
}

ll numberOfOcc(const string &s) {
    numberOfOccPreProcess();
    int cur = 0;
    for (char c: s) {
        if (st[cur].next.count(c)) {
            cur = st[cur].next[c];
        } else
            return 0;
    }
    return st[cur].cnt;
}

void dfs(int node, ll &k, string &ans) {
    k -= st[node].cnt;
    if (k <= 0) return;
    for (auto x: st[node].next) {
        if (dp[x.S] < k) {
            k -= dp[x.S];
        } else {
            ans.push_back(x.F);
            dfs(x.S, k, ans);
            return;
        }
    }
}

bool getKthLex(ll k, string &ans) {
    ans = "";
    numberOfOccPreProcess();
    if (dp[0] < k) return false;
    dfs(0, k, ans);
    return true;
}

#undef ii
#undef ll
#undef F
#undef S
};

```

KMP problem total occurrences of each prefix of the string as a substring

```
vector<int> ans(n + 1);  
for (int i = 0; i < n; i++)  
    ans[pi[i]]++;  
for (int i = n-1; i > 0; i--)  
    ans[pi[i-1]] += ans[i];  
for (int i = 0; i <= n; i++)  
    ans[i]++;
```